DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Functions

CMSC 124 Lab 5

Prepared by: Rene Andre Bedonia Jocsing

Published: November 06, 2026 **Deadline:** December 09, 2026

You have a scanner, a parser, an evaluator, and scoping. What you don't have is a way for a program to make a decision or do something more than once. This activity adds the last two pieces: **control flow** and **functions**.

Control flow lets programs choose and repeat. Functions let them bind names and reuse code. Together they make your language Turing-complete, which means it can express any computation a computer can perform. That's not a small thing to have built in one semester. This activity covers Chapters 9 and 10 of Nystrom's *Crafting Interpreters*, "Control Flow" and "Functions." It's the largest of the five, so read the whole manual before you plan the month.

Your Language, Your Rules

Lox supplies the worked examples below. Your language can use commands and mutable variables, immutable expressions and functions, or both. The required capabilities are conditional evaluation, repetition, short-circuiting logical operations, functions with argument binding and results, higher-order use, at least two native functions, and arity checking.

An **immutable binding** keeps the value it received when it was created. A functional language can introduce another binding, recurse to repeat a computation, and return the value of a function body. It doesn't need assignment, `while`, `for`, or explicit `return` statements. Implement those constructs if your design supports them. A conditional expression can produce a value where Lox uses an if statement.

Functions must be usable as values, including as arguments and results. Where your design supports escaping capture, prove that the captured bindings remain usable after the defining call ends. An immutable captured value is enough. The Lox example uses a mutable counter. An immutable design can use captured values.

If your design restricts escaping functions or doesn't capture defining bindings, document the restriction. Prove your scope rule with calls where a caller and a definition have different bindings for the same identifier. Also test repeated calls to show whether local state is fresh or persistent. Test what happens when a function outlives its defining call, including a documented rejection if that use is forbidden. Higher-order use still needs a successful test within the supported lifetime.

Follow the scope, binding, and storage rules documented in Lab 4. Static scope follows enclosing declarations. Dynamic scope follows active callers. A combination needs explicit rules about which bindings use each relation. Restore the active context on normal completion, early return where supported, and errors. Retained storage follows your lifetime policy.

Keep the run contract and the harness. Every design choice needs committed annotated test files, predicted results, and green CI. A scope diagram helps explain a result. The test proves the interpreter produces it.

Background

If-else

The most basic decision:

```
if (temperature > 30) {
    print "It's hot!";
} else {
    print "It's comfortable.";
}
```

Evaluate the condition, apply the truthiness rule you settled on in Lab 3, and execute one branch or the other. Your grammar grows:

```
statement → exprStmt | printStmt | block | ifStmt ;
ifStmt    → "if" "(" expression ")" statement ( "else" statement )? ;
```

In this Lox grammar, the branches are statements. That's what allows a single-statement branch without braces. It's also what makes `else if` work with no extra grammar: an `else` whose statement happens to be another `if`.

```
if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else {
    grade = "F";
}
```

That flexibility costs you an ambiguity. It's famous enough to have a name. The **dangling else** problem:

```
if (first)
    if (second)
        print "both";
    else
        print "???";
```

Which `if` owns the `else`? The grammar as written allows both readings. Almost every language resolves it by binding `else` to the nearest unmatched `if`. A recursive descent parser does that naturally. The inner `if` call is the one holding the parser when the `else` shows up. You get the right answer for free, which is a reason to understand why it works.

Loops

On the imperative route, while is the simpler loop:

```
var i = 0;
while (i < 10) {
```

```
    print i;
    i = i + 1;
}
```

```
statement  → ... | whileStmt ;
whileStmt  → "while" "(" expression ")" statement ;
```

Evaluate the condition, and if it's truthy, execute the body and check again. That's the entire implementation. It's the first place your interpreter can loop forever. Remember that when a test times out.

For loops add nothing to the language's power. The C-style form is **syntactic sugar**, a shorthand for a pattern you could already write:

```
for (var i = 0; i < 10; i = i + 1) {
    print i;
}
```

means

```
{
    var i = 0;
    while (i < 10) {
        print i;
        i = i + 1;
    }
}
```

```
statement → ... | forStmt ;
forStmt   → "for" "(" ( varDecl | exprStmt | ";" )
            expression? ";"
            expression? ")" statement ;
```

All three clauses are optional, so `for (;;)` is an infinite loop. The cheapest implementation is **desugaring**. Build the while-loop tree in the parser and let your existing evaluator run it, so there's no `for` logic in the evaluator at all. Note the outer block in the desugared form. It's what keeps the loop variable from leaking into the enclosing scope.

Logical Operators

Combining conditions needs `and` and `or`, or `&&` and `||` if your language uses symbols:

```
if (temperature > 30 and humidity > 80) {
    print "It's hot AND humid!";
}
```

These aren't ordinary binary operators, because they **short-circuit**. `and` evaluates its right operand only when the left is truthy. `or` evaluates its right operand only when the left is falsey. That isn't an optimization, it's semantics, and this is the example that proves it:

```
if (denominator != 0 and numerator / denominator > 1) {
    print "Ratio is greater than 1";
}
```

Without short-circuiting, a zero denominator divides by zero before the guard can stop it. With it, the right side never runs.

There's a second design question hiding here: what do these operators return? Lox returns the operand, so `nil or "hello"` is `"hello"`, which makes `or` usable for defaults. Returning a strict boolean is also fine. Decide and document.

```
expression → assignment ;
assignment → IDENTIFIER "=" assignment | logicOr ;
logicOr    → logicAnd ( "or" logicAnd )* ;
logicAnd   → equality ( "and" equality )* ;
```

`or` sits below `and` in precedence, matching how the words work in English.

Functions

Functions are the abstraction that makes programs composable. They package code for reuse, give operations names, and hide their own details.

```
fun greet(name) {
    print "Hello, " + name + "!";
}

greet("Alice");
```

```
declaration → funDecl | varDecl | statement ;
funDecl     → "fun" IDENTIFIER "(" parameters? ")" block ;
parameters  → IDENTIFIER ( "," IDENTIFIER )* ;

call        → primary ( "(" arguments? ")" )* ;
arguments   → expression ( "," expression )* ;
```

Look closely at the `call` rule. It says something less obvious than it looks:

```
call → primary ( "(" arguments? ")" )*
```

The instinct is to read a call as "a function name, followed by parentheses." The rule says something weaker and more useful. A call is a pair of parentheses applied to *whatever expression came before them*. Calling is a postfix operator, like ++ in C. The thing it operates on is any expression at all.

That's why all of these parse with no extra rules:

```
greet("Alice")          // the expression before "(" is a variable
getCallback()()         // ...is itself a call
(cond ? f : g)(1)       // ...is a parenthesized expression
counters[0]()           // ...would be an index, if your language has them
```

The `*` is what allows the chaining. Parse `primary`, and then as long as the next token is `(`, wrap whatever you have so far in a call node and keep going. `getCallback()()` becomes a call whose callee is a call, built by the same loop that handles the simple case.

You get this for free. You should let yourself have it. Writing the rule the instinctive way, as "IDENTIFIER followed by arguments," is more code, rejects programs that ought to work, and breaks the moment

you return a function from a function, which is something your own `makeCounter` example does in a few paragraphs.

In Lox, declaring a function creates a **function object** holding the parameter names, the body as an AST node, and the environment where the declaration appeared. That object goes into the environment like any other value, so a function's name is just a variable bound to it. That's what makes functions **first-class values** in your language. They can be stored in variables, passed as arguments, and returned from calls, like a number or a string. Nothing in your evaluator needs to treat them as a special category. The `call` rule above is the reason it doesn't have to.

That third field gives the Lox sketch **closures**, functions paired with captured bindings from their defining context. These bindings can remain available after the enclosing call returns. Capture can retain immutable values too.

```
fun makeCounter() {
    var count = 0;
    fun increment() {
        count = count + 1;
        return count;
    }
    return increment;
}

var counter = makeCounter();
print counter();  // 1
print counter();  // 2
```

`increment` still sees `count` after `makeCounter` has returned. The function object holds a reference to the environment it was created in. That environment remains reachable through the function. Its reclamation follows the host's ownership or garbage-collection rules. If your host language made you think about ownership in Lab 4, this is the payoff for whatever you decided.

Calling a Function

The lexical Lox sketch takes four steps. Adapt its argument binding, lookup links, and result handling to your design.

1. **Evaluate the arguments**, left to right, in the environment at the call site. They're ordinary expressions and this is ordinary evaluation.
2. **Create a fresh environment for this call**, whose enclosing environment is the function object's *captured* environment. Fresh matters here. Make a new environment per call. One per function makes recursion collapse into a single shared set of parameters.
3. **Bind each parameter name to its argument value** in that new environment, using `define` from Lab 4, since parameters are always local by definition.
4. **Execute the body** in that environment and produce the return value.

Step 2 is the one to slow down on. You have two environments available at the moment of a call. The choice between them decides what language you built.

The **captured** environment is where the function was *written*. The **call site** environment is where the function was *called from*. Parenting the new environment to the first gives you **lexical scoping**, where a name in a function body means whatever it meant at the place in the source where you can see the function. Parenting it to the second gives you **dynamic scoping**, where a name means whatever it happens to mean in whoever called you, which changes from call to call.

This program tells the two apart:

```
var greeting = "Hello";

```

```
fun greet() {
    print greeting;
}

fun rude() {
    var greeting = "Go away";
    greet();
}

rude();
```

Under lexical scoping, `greet` prints `Hello`. It was written next to the global `greeting`, so that's the one it means, and `rude`'s local variable is invisible to it no matter who calls whom. Under dynamic scoping, `greet` prints `Go away`, because at the moment of the call the innermost `greeting` belongs to `rude`.

Lexical scope makes nonlocal references depend on the surrounding source. Dynamic scope makes them depend on active callers. Choose the rule that serves your language, document its tradeoffs, and commit this distinguishing test with the expected output for that rule. A call-site link is a defect when the specification promises lexical lookup.

Return Statements

```
statement  → ... | returnStmt ;
returnStmt → "return" expression? ";" ;
```

In Lox, the expression is optional, so bare `return;` produces nil. An expression-bodied function instead produces its body value. If your language has no explicit early return, it needs no return signal. Test result propagation through nested expressions.

Return is harder than it looks. See the difficulty before you pick a technique. Consider where a `return` can legally sit:

```
fun find(target) {
    while (true) {
        if (check(target)) {
            return "found";        // three levels down
        }
    }
    return "missing";
}
```

That `return` is inside an if, inside a while, inside the function body. Your evaluator is executing it from `execute(returnStmt)`, called by `execute(ifStmt)`, called by `execute(whileStmt)`, called by the loop over the body's statements, called by the code handling the call. The two are five frames apart. The value has to get from the bottom one to the top one. Every frame in between has to stop what it was doing.

Now think about what it takes to do that with ordinary return values. `execute` would have to return something on every call, even for statements that produce nothing, and every caller would have to check whether what came back was a return or just a finished statement. The `while` loop would have to check after each statement and break out. The `if` would have to pass it up, and the statement loop would have to stop early. That's a check added to every single statement type, most of which have nothing to do with functions. Forgetting exactly one of them gives you a return that works at the top level of a function body and does nothing inside a loop. The pitfall in the list below is common because the code looks right.

So most tree-walking interpreters reach for exceptions instead. Define a return signal type carrying the value, throw it from the return statement, and catch it in the code that executes a call. Every frame

in between is unwound by the host language, which is machinery you already have and didn't have to write. It feels like abuse, since nothing has gone wrong and exceptions are supposedly for errors, but the control flow is the same: leave here, resume there, skip everything between. jlox does this.

If your host language dislikes exceptions, Rust and Go groups usually thread a signal enum through the statement executor instead, where `execute` returns something like `Normal` or `Return(value)` and each compound statement checks and propagates. That's the version described two paragraphs up, and it works fine as long as you're systematic about it. It's more code. The compiler will help you find the cases you missed. That's the reason it's tolerable in those languages and miserable in the others.

Native Functions

Native functions, also called built-ins, are implemented in your host language. They expose operations your language doesn't implement in user code, such as output or a type query. A functional language can provide pure native operations and handle output at the program boundary.

The textbook adds `clock()`, which returns the current time and exists mainly to let you benchmark your own interpreter. Others worth having: reading a line of input, converting a value to a string, or asking for a value's type.

Implement them as callable objects that run host code. Define them in the global environment before any user code runs. If you gave your language a single callable interface, a native function is just another implementation of it, and calls don't need to know the difference.

Arity Checking

A function's **arity** is how many parameters it takes. Most languages insist that calls match:

```
fun add(a, b) { return a + b; }

add(1, 2);     // fine
add(1);        // Error: Expected 2 arguments but got 1.
add(1, 2, 3);  // Error: Expected 2 arguments but got 3.
```

Check arity at call time and report a clear runtime error when it's wrong. Default parameters and variadic functions are optional extras. If you add them, arity checking gets more interesting. Your tests should show it.

Recursion

Recursion depends on how your function sees its own binding. In the lexical Lox sketch, the captured environment contains that binding. A functional language can use a recursive binding form. Test the rule your implementation uses:

```
fun fibonacci(n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

print fibonacci(10);  // 55
```

Mutual recursion means two functions call each other. Make their bindings available through declarations or a recursive binding group, and test a small terminating case.

Learning Objectives

By the end of this activity, you should be able to:

- Implement conditional evaluation and repetition using loops or recursion
- Explain the grammar and evaluation rules for the control forms your language supports
- Demonstrate short-circuiting with a test that would fail if the skipped operand ran
- Represent functions as values and explain argument binding, results, and higher-order use
- Implement early-return propagation where supported, or result propagation through expression bodies
- Explain captured bindings or the restrictions on escaping functions in your design
- Add native functions and apply arity checks to native and user functions
- Defend your chosen scope and storage rules through implementation traces and harness tests
- Write deterministic control-flow and recursion tests that stay fast enough for every push

---

Task

Finish the language.

Extend your interpreter with conditionals, repetition through loops or recursion, short-circuiting logical operators, function creation and calls, higher-order use, results, at least two native functions, and arity checking. Add assignment, imperative loops, escaping capture, and explicit return according to your documented design. Plain `./run <file>` executes programs, as in Lab 4. All three earlier flags stay, and `tests/lab0` through `tests/lab4` have to keep passing.

You'll also write a non-trivial program in your own language and commit it, which is the first time this semester the interesting artifact is written in your language.

Required Features

Grammar Extension

Add rules for your conditional forms, short-circuiting logical operators, function creation, calls, and results. Add loop and return syntax if your language supports them. Calls can use the postfix grammar shown above or your documented alternative. Update the Grammar section of your `README.md` so it matches the parser.

Control Flow

Implement conditional evaluation that selects only the required branch, and short-circuiting `and` and `or` or their equivalents. Test nested conditionals and every branch. If your syntax permits a missing alternative branch, document and test its result.

Implement repetition through loops or recursion. If you support Lox-style `while` and `for`, test both and explain any desugaring. A functional language can instead test a recursive computation with a base case and a recursive case. Scope and state lifetime must follow the Lab 4 specification in both routes.

Functions

Implement function creation, argument evaluation and parameter binding, calls, function results, higher-order use, and arity checking with a clear message. Use named declarations or anonymous function expressions according to your grammar. A function must be passable as an argument and producible as a result within your supported lifetime rules.

If explicit early returns exist, prove they propagate through nested control forms. Otherwise prove nested expression results reach the caller. Test recursive and mutually recursive calls with small inputs.

If functions capture escaping bindings, test retained values and independent captures. Immutable factories can produce adders with different captured values. Mutable factories can produce counters with separate state. If escaping capture is unsupported, submit the scope, repeated-call lifetime, and escape-restriction tests described above. These alternatives receive equal consideration when the implementation matches the specification.

Native Functions

Add at least two, defined in the global environment before user code runs. A clock is the conventional first one. A line reader, a string conversion, or a type query are all reasonable second choices. Document each one's name, arity, and return type in your `README.md`, because they're part of your language's surface now.

Document whether output uses a statement, a native function, or the program-result convention from Lab 4. Pure native functions such as conversion and type queries can meet the native-function requirement without mutation.

Runtime Errors

Calling something that isn't callable and calling with the wrong number of arguments are runtime errors: message with a line number on stderr, exit 70 in file mode, prompt returns in the REPL.

Deep recursion is the interesting case. `fibonacci(10000)` will exhaust your host language's call stack, and what happens next is your choice. A clean runtime error about stack depth is much nicer than a host stack trace, though catching it portably is awkward in some hosts. If you can't catch it, say so in your `README.md` under known limitations. An honest limitation is a better answer at defense than a surprise.

The Showcase Program

Write at least one non-trivial program in your own language and commit it under `examples/`. Non-trivial means it uses functions and control flow to do something with a point: a small simulation, a sorting routine, a text-processing pass, a game of some kind, an interpreter for something even smaller. Recursive Fibonacci is a test.

This is also the artifact that shows whether the language you designed in Lab 1 is pleasant to write in. If writing fifteen lines in your own language is annoying, say so in your `README.md`. That earns more credit than pretending otherwise.

The Run Contract

Unchanged from Lab 4, and unchanged for the rest of the semester:

```
./run           tests/lab5/closures.mylang        # execute a program
./run --tokenize tests/lab1/keywords.mylang       # Lab 1
./run --parse    tests/lab2/precedence.mylang     # Lab 2
./run --eval     tests/lab3/arithmetic.mylang     # Lab 3
```

Exit 0 when the program ran, 65 when you rejected it before running anything, 70 when it started and hit a runtime error. Program output on stdout, diagnostics on stderr. Four entry points, four folders of tests that still pass, and one build script that produces all of it from a clean checkout.

Writing Tests

Inline mode, no flag, same as Lab 4. `tests/lab5/manifest.json`:

```
{
  "ext": ".mylang",
  "mode": "inline",
  "comment_prefix": "//"
}
```

The functional route uses the same manifest and annotations. These examples use illustrative expression syntax with `let rec` for recursive bindings, `fun` for anonymous functions, and a native `print`. Translate them to your grammar. If file mode displays a program result, use that convention to expose the same values.

`tests/lab5/functional-control.mylang` checks conditional results and recursive repetition:

```
let rec sum = fun(n) ->
    if n == 0 then 0 else n + sum(n - 1)
in {
    print(sum(0));                      // expect: 0
    print(sum(4));                      // expect: 10
    print(if false then 99 else 7);      // expect: 7
}
```

`tests/lab5/immutable-capture.mylang` checks higher-order use and independent immutable captures:

```
let makeAdder = fun(base) -> fun(x) -> base + x in
let apply = fun(f, x) -> f(x) in
let addTwo = makeAdder(2) in
let addTen = makeAdder(10) in {
    print(apply(addTwo, 3));             // expect: 5
    print(apply(addTen, 3));             // expect: 13
    print(apply(addTwo, 3));             // expect: 5
}
```

`base` remains available after `makeAdder` finishes. The two returned functions retain different bindings. Repeating the first call leaves its captured value unchanged. A design that forbids this escape instead submits successful higher-order use within a live scope and an annotated rejection for the forbidden use.

A pure test can expose short-circuiting through an expression that would raise an error if evaluated. Suppose your language rejects adding a string to a number, as documented and tested in Lab 3:

```
print(false and ("bad" + 1));            // expect: false
print(true or ("bad" + 1));              // expect: true
```

Use an operation that your language really rejects at runtime. A language that rejects these operands statically can use a well-formed call whose body fails only when evaluated, or another valid runtime-failing expression. Choose operands so each logical operator's expected result follows your specification. The test must succeed when the right operand is skipped and fail if it runs.

Control flow is where inline annotations get elegant. The expected output sits right next to the branch that produces it. `tests/lab5/loops.mylang`:

```
var i = 0;
while (i < 3) {
    print i;      // expect: 0
                  // expect: 1
                  // expect: 2
    i = i + 1;
}
```

Note what happened there. The `expect:` lines are matched against stdout in file order. They don't have to be attached to the statement that produced them. A loop body prints three lines, so three annotations sit together, and the harness compares the sequence.

The imperative route can expose short-circuiting with a side effect:

```
fun sideEffect() {
    print "evaluated";
    return true;
}

false and sideEffect();   // right side must not run
print "done";             // expect: done
```

If your `and` doesn't short-circuit, `evaluated` appears on stdout, the sequence no longer matches, and the test fails. That test can only fail for the reason it was written for. That's what you want.

Mutable lexical closures, the same way:

```
fun makeCounter() {
    var count = 0;
    fun increment() {
        count = count + 1;
        return count;
    }
    return increment;
}

var counter = makeCounter();
print counter();   // expect: 1
print counter();   // expect: 2
print counter();   // expect: 3

var other = makeCounter();
print other();     // expect: 1
```

That last pair of lines is the test most groups forget. It's the one that catches a captured environment shared between calls. In this counter design, each factory call needs distinct captured state. A static-local design instead tests its documented shared state.

Errors, one per file:

```
fun add(a, b) { return a + b; }
add(1);            // expect runtime error: Expected 2 arguments but got 1.
```

Two Cautions Specific to This Activity

Keep tests fast. The harness kills any test file after **15 seconds**. A tree-walking interpreter computing naive recursive Fibonacci is slow enough that the number you pick matters. `fibonacci(10)` returns instantly, `fibonacci(20)` is fine, and somewhere past that you're timing your host language. If you want a benchmark, put it in `examples/` and run it by hand. This is what `clock()` is for.

Keep nondeterminism out of expectations. A test asserting what `clock()` returns fails on the second run. If you want to test a native clock at all, assert something stable about it, such as comparing two readings and printing whether time moved forward. Anything whose value depends on the machine, the moment, or the map iteration order belongs outside your expectation-checked tests.

Run every folder locally before pushing:

```
curl -sSL https://raw.githubusercontent.com/WhiteLicorice/cmsc-124-harness/v1.1/run_tests.py
  ↪  -o run_tests.py
./build.sh
python run_tests.py tests/lab0
python run_tests.py tests/lab1
python run_tests.py tests/lab2
python run_tests.py tests/lab3
python run_tests.py tests/lab4
python run_tests.py tests/lab5
```

Use `python3` on Linux and macOS, and Git Bash on Windows.

Continuous Integration

```
      - run: python3 run_tests.py tests/lab0
      - run: python3 run_tests.py tests/lab1
      - run: python3 run_tests.py tests/lab2
      - run: python3 run_tests.py tests/lab3
      - run: python3 run_tests.py tests/lab4
      - run: python3 run_tests.py tests/lab5
```

Six folders, all green, before you book a defense. By now, that workflow is the closest thing you have to a specification of your language's behavior. It took you four months to write. It's also the reason you can add functions in November without wondering whether you broke the scanner in August.

Implementation Notes

One Callable Interface

Define a single interface or trait for anything that can be called, with a method that takes the interpreter and the evaluated arguments and returns a value, plus a way to ask its arity:

```
interface Callable:
    arity()                        // how many parameters
    call(interpreter, arguments)   // do the thing, return a value
```

The lexical Lox sketch implements it twice. Once for user-declared functions, which walk an AST body:

```
UserFunction implements Callable:
    declaration     // the AST node: name, parameters, body
    closure         // the environment where it was declared

    function arity():
        return declaration.parameters.length

    function call(interpreter, arguments):
        environment = new Environment(enclosing: closure)   // NOT the call site
        for i in 0 .. arguments.length:
            environment.define(declaration.parameters[i].name, arguments[i])
        try:
            interpreter.executeBlock(declaration.body, environment)
        catch ReturnSignal as r:
            return r.value                                  // the return landed here
        return nil                                          // fell off the end
```

And once for natives, which run host code instead:

```
NativeClock implements Callable:
    function arity(): return 0
    function call(interpreter, arguments):
        return currentTimeInSeconds()      // host language does the work
```

Now the code that evaluates a call node never branches on which kind it's holding:

```
function evaluateCall(node):
    callee = evaluate(node.callee)
    arguments = [evaluate(arg) for arg in node.arguments]

    if callee is not a Callable:
        throw RuntimeError(node.paren, "Can only call functions.")
    if arguments.length != callee.arity():
        throw RuntimeError(node.paren,
            "Expected " + callee.arity() + " arguments but got " + arguments.length + ".")

    return callee.call(this, arguments)
```

Arity checking happens in one place, the not-callable check happens in one place, and adding a native later is a few lines that touch nothing else. Skipping this abstraction is the most common source of duplicated logic in this activity. It shows up as arity being checked for user functions but not for natives, which is a bug nobody finds until defense.

Return as a Signal

This sketch applies when your language supports explicit early returns. An expression-bodied language evaluates its body and passes the resulting value to the caller.

Define a return signal type carrying a value. Throw or return it from the return statement's execution, and catch or intercept it in the code that executes a function body:

```
ReturnSignal { value }                  // not an error, just a courier

function executeReturn(node):
```

```
    value = node.expression exists ? evaluate(node.expression) : nil
    throw ReturnSignal(value)             // unwinds however many frames it takes
```

The only place that catches it is `call` in the sketch above. Nothing between the throw and the catch knows returns exist. That's the entire point.

If your host language has exceptions, disable stack trace capture on that type if it's cheap to do, since you'll be throwing it on every single call and the trace is pure waste.

Closures and the Environment Chain

In the lexical Lox sketch, a function object stores the environment active where it was declared. Calling it creates a new environment whose parent is that stored one. It's the `closure` field and the `enclosing:` argument in the `UserFunction` sketch above. An immutable design can capture the values it needs or retain immutable environments. The environment survives because the function object holds a reference to it, so reclamation follows the host's ownership rules. Reference-counted implementations also need to account for cycles.

Closures capture the environment, so several closures created in the same scope share what they see. Loop variables make that visible. Whether closures created inside a loop body each see their own copy of the loop variable depends on whether your desugaring gives each iteration a fresh environment. Both behaviors exist in other languages. JavaScript's `var` and `let` differ on this point, so test what yours does and write it in the README.

For-Loop Desugaring

If your language supports this imperative loop form, desugar it in the parser unless your design needs direct evaluation. Parse the three clauses, build a while node whose body is a block containing the original body and the increment, then wrap the whole thing in a block containing the initializer. The evaluator needs no `for` case at all, which means a bug in your loops can only be in one place.

Common Pitfalls

- Lookup following callers when the specification promises lexical scope, or enclosing declarations when it promises dynamic scope
- Logical operators evaluating a supposedly skipped operand
- Early returns stopping at an intermediate loop or block when explicit returns are supported
- Nested expression results failing to reach the caller
- Captured bindings disappearing before the lifetime promised by the specification ends
- Immutable bindings changing through accidental mutation
- Native and user functions receiving different arity checks
- A non-callable value crashing the host instead of producing a runtime error
- Loop bindings or recursive-call bindings escaping the scope promised by the design
- Infinite loops or recursion exceeding the 15-second test timeout
- Nondeterministic expectations built on a clock or other changing value
- Breaking an earlier test folder while extending the parser

Testing Strategy

Your `tests/lab5/` folder must cover these capabilities with annotated programs:

- Conditionals selecting each branch, including nested conditionals
- Repetition through a loop or recursion, including zero repetitions or the recursive base case

- Short-circuiting for both logical operators, plus their documented result values
- Calls with no arguments and several arguments
- Function results reaching the caller through nested control or expression forms
- Recursion and mutual recursion with small terminating inputs
- A function passed as an argument and a function produced as a result within the supported lifetime
- Each native function with stable expectations

For supported imperative constructs, add while and for cases, empty clauses where legal, and early return from nested loops and conditionals. For expression-bodied functions, add nested conditional-result cases. Test a missing return or body result only when the grammar permits it.

For escaping capture, test two independent captures from one factory. Use mutable counters or immutable adders according to your design. Test loop capture only if loops exist. If escaping capture is unsupported, test lookup where caller and definition disagree, repeated-call state lifetime, and the documented behavior when a function outlives its defining call.

Then test errors, one per file: too few arguments, too many arguments, and calling a non-callable value. Add illegal returns, forbidden immutable updates, and forbidden escaping uses where the grammar can express them. Match static rejection with `expect error:` and runtime failure with `expect runtime error:`.

Include interaction tests, such as recursive calls inside a conditional or a higher-order call using captured bindings. Derive the output before running the harness. All six lab folders must pass. Diagrams and explanations support these files and cannot replace them.

Deliverables

Your group's GitHub repository, presented during appointments, with a clean incremental commit history showing who wrote what, must contain:

1. Control flow and function implementation, including shared call handling, your capture or escape policy, and function-result handling, documented where the code doesn't explain itself
2. A working scanner, parser, evaluator, and environment, with all three earlier flags intact
3. `build.sh` and `run` at the root, executable, where plain `./run <file>` executes a program and `./run` starts a REPL
4. `tests/lab5/` with `manifest.json` and your annotated test files
5. `.github/workflows/test.yml` running all six lab folders, green on the commit you defend
6. A `README.md` complete against the specification template: the finished grammar, the native function table, your paradigm, scope, storage, mutability, evaluation, logical-result, function-result, and capture decisions, an honest list of known limitations, and a changelog entry per activity
7. At least one non-trivial example program under `examples/`, plus a note in the README on how to run it

Then, each member submits, individually after the laboratory defense through email: a short `reflection.txt`, this time covering the whole semester's work, plus a short `peer.txt` with your honest assessment of how your groupmates, including yourself, worked during the activity. Adhere to the following subject line: `[CMSC 124 Lab] Lab 5: LastName, Initials`, for example: `[CMSC 124 Lab] Lab 5: Sanchez, SM`. Include a link to your group's GitHub repository in the email. If even one member of a group fails to submit their individual deliverables, no final grade for the activity may be released for all members of the group.

This is the last activity of the semester. Mind the syllabus end date. After December 9, 2026, I can no longer accept laboratory deliverables, and a missing deliverable churns out either your grade as it stands or an INC where that applies.

Academic Honesty

Using large language models to generate wholesale vibe-coded submissions is cheating. It's subject to failure in the course and harsh disciplinary action. Function lookup and lifetime are easy to copy without understanding. Expect to trace which bindings a call sees, where its result comes from, and why its storage is still valid.

Important Dates

Progress reports and the laboratory defense may be booked only during the dates and hours defined in the syllabus. Book ahead through the booking page on the course site.

Activity	Monday	Tuesday
Week 1 Progress Report	Nov 9	Nov 10
Week 2 Progress Report	Nov 16	Nov 17
Week 3 Progress Report	Nov 23	Nov 24
Week 4 Laboratory Defense	**Nov 30**	**Dec 1**

Appointments need my verification before they count. If a date falls on a holiday or a suspension of classes, the syllabus allows a recorded progress report or defense for that seven-day period. Nov 30 is Bonifacio Day. Plan your defense for Dec 1. Book it early. It's the last defense slot of the semester and every group in the class needs one.

For progress reports, Week 1 wants conditionals and the repetition route in the updated grammar, with conditional harness tests and a loop or recursive-base-case test. Week 2 wants working repetition, logical operators, function creation and calls, and an attempt at capture or the alternative lifetime evidence. Week 3 wants function results, higher-order use, capture or escape-policy tests, natives, arity checking, and green CI across all six folders, plus your example program committed.

This is the last defense window of the semester. The margin after it's thin. A group that reaches the last week of November with function behavior still broken has no fifth week to fall back on, which is the whole argument for the prescribed timeline, made one final time.

Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Implementation Correctness (25%)	Every required capability works under the documented language design, including edge cases; the run contract is honored exactly, with program output on stdout, diagnostics on stderr, and the right exit code for each class of failure.	Core features correct with minor gaps in fringe cases; streams and exit codes right.	Core features work, but several requirements are missing or wrong; stream separation or exit codes are inconsistent.	Fails the activity's core requirement, ignores the run contract, or crashes with host-language errors on malformed input.
Testing and CI (20%)	Committed tests cover every required feature and its failure cases; expectations were reasoned out; every lab folder is green in CI on the defended commit.	Good coverage of the main features with at least one failure case; CI green.	Only happy paths tested, or an earlier lab folder left failing.	Almost no tests, no manifest, or no working workflow.
Code Engineering (10%)	Clean separation between pipeline stages; shared logic written once; constructs idiomatic for the host language; comments explain why.	Mostly clean structure with minor duplication or naming problems.	Monolithic functions; repeated logic; one stage leaking into another.	Unreadable, or the code contradicts the documented language design.
Collaboration (20%)	Professional Git usage: atomic, semantic commits from every member spread across the accomplishment period; tests and features arrive together; evidence of review or pair programming.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history, or history was rewritten or crammed.
Technical Defense (25%)	Every member traces their own code over fresh input, handles what-if questions, and defends the language design decisions behind the implementation.	Clear explanation of the implementation; both members participate meaningfully; logic delivery is sound.	Unclear explanations; uneven participation; struggles with code-tracing questions.	Unprepared; cannot explain the pipeline or defend design decisions.